

Emotion Recognition Project Documentation

Welcome to the documentation for the Emotion Recognition Project! This system combines deep learning and classical machine learning to perform real-time emotion classification from facial images. It is designed for robustness, explainability, and ease of extension.

Project Highlight

This project integrates CNN-based feature extraction with AdaBoost and SVM classifiers for accurate, real-time facial emotion recognition, including comprehensive explainability using SHAP.

Project Files Overview

Below is a summary of each file and its role in the project.

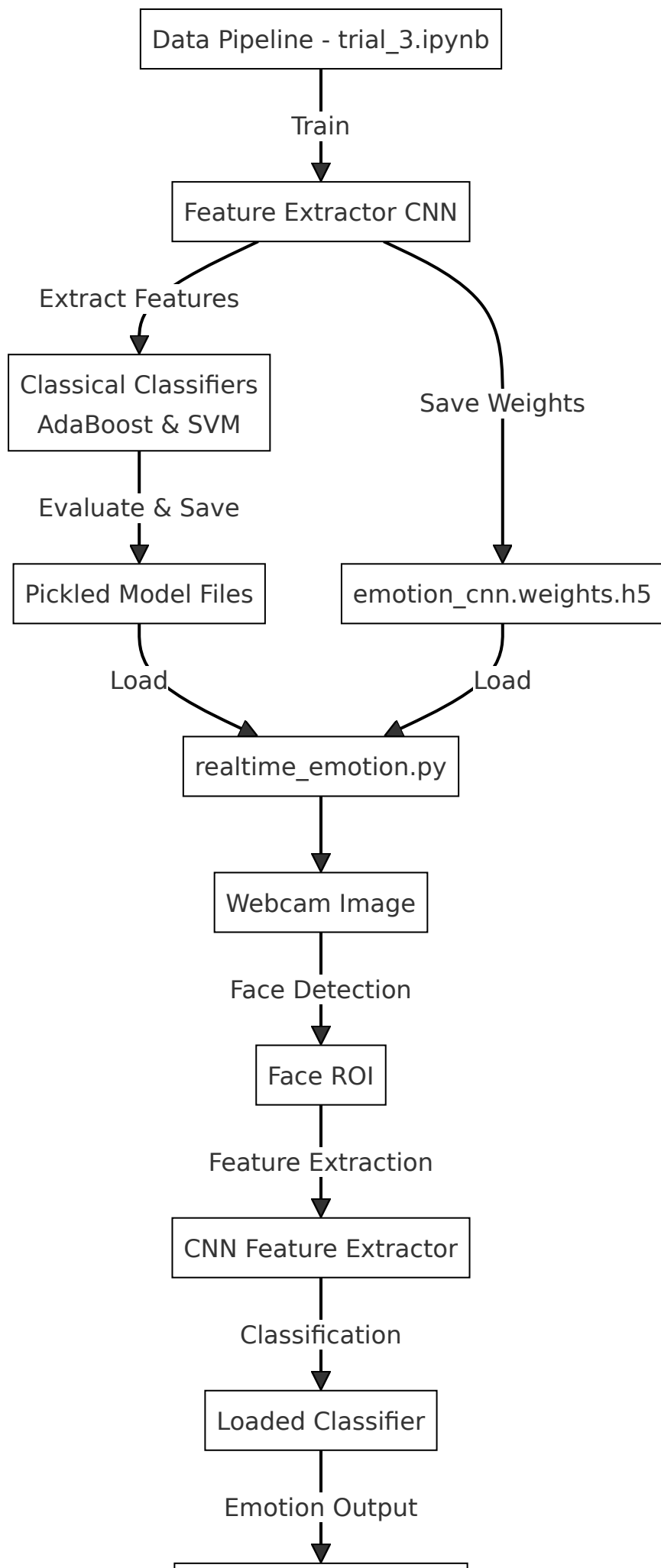
Filename	Description
<code>.gitignore</code>	Prevents version control of the Python virtual environment (<code>venv/</code>).
<code>requirements.txt</code>	Lists all required Python packages for reproducibility.
<code>trial_3.ipynb</code>	Main Jupyter notebook for data processing, model training, evaluation, SHAP analysis
<code>realtime_emotion.py</code>	Standalone script for running real-time emotion detection via webcam.
<code>adaboost_emotion_classifier.pkl</code>	Pickled AdaBoost classifier (trained).
<code>svm_emotion_classifier.pkl</code>	Pickled SVM classifier (trained).

System Architecture

The system has three main phases:

- Data Preparation & Model Training** (in `trial_3.ipynb`)
- Model Serialization** (weights and pickled classifiers)
- Real-Time Recognition** (in `realtime_emotion.py`)

Component Flow



1. `.gitignore`

This file ensures that files and directories matching the pattern `venv/` are ignored by git. The virtual environment directory contains third-party dependencies and machine-specific files that should not be tracked in the repository.

```
1 venv/
```

2. `requirements.txt`

This file lists all Python dependencies needed to run the project. It should be installed via:

```
1 pip install -r requirements.txt
```

Contents:

```
1 pandas
2 scikit-learn
3 numpy
4 matplotlib
5 keras
6 tensorflow
7 ipykernel
```

- **pandas, numpy:** Data manipulation.
- **scikit-learn:** Classical ML, model persistence, metrics.
- **matplotlib:** Data visualization, SHAP plots.
- **keras, tensorflow:** Deep learning and CNN feature extractor.
- **ipykernel:** Jupyter notebook support.

3. `trial_3.ipynb` – Model Development & Explainability

This Jupyter notebook is the heart of your project. It covers data preprocessing, CNN feature extractor training, classifier training (AdaBoost and SVM), model evaluation, and model explainability using SHAP. It also serializes the trained models for deployment.

Key Steps in the Notebook

3.1. Import Libraries

```
1 import numpy as np
2 import pandas as pd
3 import shap
4 import matplotlib.pyplot as plt
5 from sklearn.model_selection import train_test_split
6 from sklearn.svm import SVC
7 from sklearn.ensemble import AdaBoostClassifier
8 from sklearn.metrics import accuracy_score
9 from keras.models import Sequential, Model
10 from keras.layers import Conv2D, MaxPooling2D, Dropout, Flatten, Dense
11 from keras.utils import to_categorical
```

3.2. Data Loading and Preprocessing

- Loads a dataset (e.g., `ckextended.csv`) with columns: `emotion`, `pixels`.
- Handles missing file with dummy data for demonstration.
- Drops the `'Usage'` column if it exists.
- Converts pixel strings to normalized 48×48 images.

Example code:

```
1 def process_pixels(pixels_series):
2     images = []
3     for pixel_seq in pixels_series:
4         pixel_vals = np.array([int(p) for p in pixel_seq.split()], dtype='float32')
5         image = pixel_vals.reshape(48, 48, 1) / 255.0
6         images.append(image)
7     return np.array(images)
8
9 x_data = process_pixels(data['pixels'])
10 y_data = data['emotion'].values
11 y_cat = to_categorical(y_data, num_classes=8)
```

3.3. Train-Test Split

Performs stratified split to preserve emotion class distribution.

```
1 x_train, x_test, y_train_cat, y_test_cat, y_train_labels, y_test_labels = train_test_split(
2     x_data, y_cat, y_data, test_size=0.2, random_state=42, stratify=y_data
3 )
```

3.4. Define and Train the CNN Feature Extractor

The CNN architecture:

- Input: (48,48,1) grayscale facial image
- Conv2D/MaxPooling/Dropout layers
- Flatten + Dense(128, relu) as "feature_layer"
- Final Dense(8, softmax) for classification (not used for deployment)

Example Keras summary:

```
1  cnn_model = Sequential([
2      Conv2D(64, kernel_size=(3, 3), activation='relu', input_shape=(48, 48, 1)),
3      MaxPooling2D(pool_size=(2, 2)), Dropout(0.25),
4      Conv2D(128, kernel_size=(3, 3), activation='relu'),
5      MaxPooling2D(pool_size=(2, 2)), Dropout(0.25),
6      Conv2D(256, kernel_size=(3, 3), activation='relu'),
7      MaxPooling2D(pool_size=(2, 2)), Dropout(0.25),
8      Flatten(), Dense(128, activation='relu', name='feature_layer'),
9      Dropout(0.5), Dense(8, activation='softmax')
10 ])
11 cnn_model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

Training:

- 100 epochs, batch size 64
- Validation on the test split

Model Saving:

```
1  cnn_model.save_weights('emotion_cnn.weights.h5')
```

3.5. Feature Extraction

After training, a `feature_extractor` Keras `Model` is defined to output the 128-dimensional feature vector:

```
1  feature_extractor = Model(
2      inputs=cnn_model.layers[0].input,
3      outputs=cnn_model.get_layer('feature_layer').output
4  )
5  x_train_feat = feature_extractor.predict(x_train)
6  x_test_feat = feature_extractor.predict(x_test)
```

3.6. Train and Compare AdaBoost and SVM Classifiers

- Both classifiers are trained on the 128-dimensional features.
- AdaBoost uses 100 decision stumps; SVM uses linear kernel with probability outputs for SHAP.

AdaBoost:

```
1  ada = AdaBoostClassifier(n_estimators=100, random_state=42)
2  ada.fit(x_train_feat, y_train_labels)
```

SVM:

```
1  svm = SVC(kernel='linear', probability=True, random_state=42)
2  svm.fit(x_train_feat, y_train_labels)
```

Accuracy:

AdaBoost Accuracy on CNN features: 0.8804

SVM Accuracy on CNN features: 0.9348

3.7. Model Explainability with SHAP

Global Feature Importance (Bar Plot)

- SHAP KernelExplainer for both classifiers.
- Background data is summarized with KMeans.
- Plots show which features (CNN-learned) are most impactful across the test set.

Example:

 AdaBoost SHAP Bar Plot Example

Local Explanation (Force Plot)

- For a single sample, visualizes how each feature contributed to the predicted emotion.

 AdaBoost SHAP Force Plot Example

Beeswarm Plot

- Shows the distribution of SHAP values for each feature across samples and classes.
- Helps you see which features are consistently positive/negative for certain emotions.

 AdaBoost SHAP Beeswarm Example

SVM SHAP Analysis

- Identical explainability workflow is repeated for the SVM classifier.

3.8. Model Serialization

At the end, the notebook saves all artifacts for later use:

- CNN weights: `emotion_cnn.weights.h5`
- AdaBoost: `adaboost_emotion_classifier.pkl`
- SVM: `svm_emotion_classifier.pkl`

Example:

```
1 import joblib
2 joblib.dump(ada, 'adaboost_emotion_classifier.pkl')
3 joblib.dump(svm, 'svm_emotion_classifier.pkl')
```

4. `realtime_emotion.py` – Real-Time Emotion Detection

This script provides the runtime interface to use the trained models for real-time facial emotion recognition from a webcam.

Key System Steps

4.1. Dependencies

- `cv2`: For webcam capture and drawing.
- `numpy`, `joblib`, `keras`, `tensorflow`, `requests`, `os`

4.2. Constants and Model Paths

- **EMOTION_LABELS:** Maps integer class to emotion name.
- **Paths:** To CNN weights, classifier pkl, Haar cascade XML.

4.3. Haar Cascade Download

If the OpenCV face detector XML is missing, it is downloaded automatically.

```
1 def download_haar_cascade():
2     """Downloads the Haar Cascade XML file if it doesn't exist."""
3     if not os.path.exists(HAAR_CASCADE_PATH):
4         # Download logic...
5     return cv2.CascadeClassifier(HAAR_CASCADE_PATH)
```

4.4. CNN Feature Extractor Reconstruction

- CNN is rebuilt with same architecture as in training.
- Weights loaded from `emotion_cnn.weights.h5`.
- Only the feature extraction up to the `'feature_layer'` is used.

4.5. Classifier Loading

- Loads AdaBoost classifier with `joblib`.

4.6. Main Real-Time Loop

- Opens webcam (`cv2.VideoCapture(0)`).
- Detects faces in grayscale frames.
- For each face:
 - Extracts ROI, resizes to 48×48, normalizes.
 - Passes through CNN feature extractor to get 128-dim feature.
 - Classifier predicts emotion label.
 - Draws rectangle and label on the frame.

Core snippet:

```
1 ret, frame = video_capture.read()
2 gray_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
3 faces = face_cascade.detectMultiScale(gray_frame, ...)
4 for (x, y, w, h) in faces:
5     # Preprocess
6     # CNN feature extraction
7     # Classifier prediction
8     cv2.putText(frame, predicted_label, ...)
9 cv2.imshow('Emotion Detector', frame)
```

- Press `q` to exit.

5. Model Files

5.1. `adaboost_emotion_classifier.pkl`

A binary file storing a scikit-learn AdaBoost classifier, trained on CNN features for emotion recognition.

- Loaded in both `trial_3.ipynb` and `realtime_emotion.py` for prediction.

5.2. `svm_emotion_classifier.pkl`

A binary file storing a linear SVM classifier (with probability estimates), also trained on CNN features.

Note: While the real-time script defaults to AdaBoost, SVM can be loaded similarly by changing `CLASSIFIER_PATH` and ensuring the interface matches.

6. Best Practices & Recommendations

Production & Extensibility Tips

For best results, ensure Haar cascades and models are present at runtime. You can easily swap AdaBoost for SVM by changing only the classifier load path. To extend to new emotions, retrain the CNN and classifiers with updated labels.

7. Example Usage

7.1. Model Development and Explanation

1. Run `trial_3.ipynb` cell by cell.
2. Visualize SHAP plots for feature importance and local explanation.
3. Save weights and pickles.

7.2. Real-Time Demo

After installing dependencies and saving all models:

```
1 python realtime_emotion.py
```

- The webcam window will show live predictions above detected faces.

8. Interactive API Blocks

There are no HTTP API endpoints in the provided code, so no API block is generated. The system is built for in-memory or script-based usage.

9. Frequently Asked Questions (FAQ)

Q: How do I switch between AdaBoost and SVM for real-time detection?

A: In `realtime_emotion.py`, change `CLASSIFIER_PATH` to point to `svm_emotion_classifier.pkl` and retrain the feature extractor as needed.

Q: What does the CNN feature vector represent?

A: It is a learned, 128-dimensional embedding of facial features, extracted by the trained convolutional network.

Q: Why use SHAP?

A: SHAP provides both global and local explanations for model predictions, critical for understanding which features most impact emotion classification.

10. Visual Examples

CNN Feature Extractor Architecture

 CNN Architecture

Real-time Webcam Demo

 Webcam Demo Example

11. Code Snippet: Real-Time Prediction Loop

```
1 while True:
2     ret, frame = video_capture.read()
3     if not ret:
4         break
5     gray_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
6     faces = face_cascade.detectMultiScale(gray_frame, ...)
7     for (x, y, w, h) in faces:
8         # Preprocessing...
9         image_features = feature_extractor.predict(reshaped_face, verbose=0)
10        prediction = classifier.predict(image_features)
11        predicted_label = EMOTION_LABELS.get(prediction[0], "Unknown")
12        cv2.putText(frame, predicted_label, (x, y - 10), ...)
13    cv2.imshow('Emotion Detector', frame)
14    if cv2.waitKey(1) & 0xFF == ord('q'):
15        break
```

12. Extending and Customizing

- **New Dataset:** Replace `ckextended.csv` with your own dataset, ensuring same format (`emotion` , `pixels`).
- **Emotions:** Change `EMOTION_LABELS` and retrain models as needed.
- **Model Tuning:** Hyperparameters (batch size, layers, epochs) can be adjusted in `trial_3.ipynb`.
- **Explainability:** SHAP workflow can be extended to new classifiers or features.

13. Conclusion

This Emotion Recognition system demonstrates a robust and interpretable approach using CNN-based features and classical ML classifiers. The project supports explainability, real-time deployment, and easy extensibility for new data or models.

Documentation Complete

Your project is now fully documented! For issues or contributions, update the README and open issues as appropriate.

Happy coding and emotion detecting!